ToolCrib Software

Software License Architecture, Hardware Fingerprint & Verification Guide

System Security Architecture Overview:
ToolCrib employs a robust dual-layer license protection architecture combining **RSA 3072-bit asymmetric digital signatures**, **SHA-256 machine hardware fingerprinting**, and **Ed25519 Base64URL token encryption**. This specification explains the hardware attributes collected, the generation algorithms for Fingerprints and Signatures, and the 5-gate verification mechanism implemented across backend and frontend UI.

1. System Hardware Information Attributes

To prevent unauthorized license sharing, software copying, or virtual machine cloning, ToolCrib extracts 5 core hardware identifiers from the underlying Windows operating system environment (via WMI, CIM, and native Python system calls in License/client_license.py):

Hardware Attribute	Extraction Method / Command	Purpose & Description
Primary MAC Address	psutil.net_if_addrs() or uuid.getnode()	Primary network interface card MAC address (12-char hex, e.g. 0068EBBCA085). Binds license to network hardware.
System UUID	Get-CimInstance Win32_ComputerSystemProduct .UUID	Universally Unique Identifier generated by motherboard vendor / BIOS. Highly persistent hardware identifier.
Motherboard Serial	Get-CimInstance Win32_BaseBoard .SerialNumber	Serial number physically etched onto main system motherboard.
BIOS Serial Number	Get-CimInstance Win32_BIOS .SerialNumber	System BIOS firmware serial number issued by OEM manufacturer (Dell, HP, Lenovo, etc.).
Physical Disk Serials	Get-CimInstance Win32_DiskDrive .SerialNumber	Sorted comma-separated list of physical hard drive / NVMe drive serial numbers.

2. Hardware Fingerprint Formulation & Purpose

A **Hardware Fingerprint** is a unique 64-character SHA-256 hexadecimal digest that uniquely identifies the host computer. It acts as a cryptographic hardware lock.

Fingerprint Assembly Formula:
1. Combine harvested hardware fields using vertical bar delimiters:
   raw_string = MAC | SystemUUID | MotherboardSerial | BiosSerial | DiskSerials
2. Convert all strings to uppercase.
3. Compute SHA-256 hash: hashlib.sha256(raw_string.encode('utf-8')).hexdigest().upper()

Example Output Fingerprint (64 Characters):
3B9E512C69514D190FF4836F2E5C0E578B85266AFBE1714DAE42D77285536D3D

Purpose & Security Role:
• **Anti-Cloning Protection:** If an authorized user copies the database or software installation to another computer, the new host machine will produce a completely different SHA-256 fingerprint, instantly invalidating the license.
• **Privacy-Preserving:** Hardware serials are hashed into a one-way digest, ensuring sensitive component serial numbers are never exposed in plain text.

3. RSA Digital Signature Formulation & Purpose

While the *Fingerprint* locks the software to host hardware, the **RSA Digital Signature** locks the license terms (License ID, Product, Expiration Date, and Fingerprint) against modification or forgery.

Signature Generation Workflow (Vendor Side - License/generate_license.py):
1. Build canonical data string: data_to_sign = f'{license_id}|{fingerprint}|{product}|{expiry_str}'
2. Sign string using vendor's RSA 3072-bit Private Key (PKCS1v15 padding + SHA-256 hashing).
3. Encode resulting signature bytes into Base64 string.

Signature Verification Workflow (Client Side - License/client_license.py & Backend):
1. Read RSA Public Key (public_key.pem).
2. Re-assemble canonical string LicenseId|Fingerprint|Product|Expiry from JSON fields.
3. Call public_key.verify(signature_bytes, data_bytes, padding.PKCS1v15(), hashes.SHA256()).

Purpose & Security Role:
• **Tamper Prevention:** If a user opens license_TC-10001.json and changes Expiry from 2026 to 2035, or alters the Fingerprint, the RSA signature verification fails immediately because only the vendor possesses the private key.
• **Non-Repudiation & Authenticity:** Proves the license was legitimately issued by ToolCrib software vendor.

4. RSA License JSON Key Structure

When python License/generate_license.py is executed, it produces the following JSON structure containing all required activation tokens:


```
{ "LicenseId": "TC-10001", "Fingerprint":
"3B9E512C69514D190FF4836F2E5C0E578B85266AFBE1714DAE42D77285536D3D", "Product": "ToolCrib", "Expiry":
"2027-10-05T04:17:31.026283Z", "Signature":
"JecG6j6L9i89TnjUB8c6TqOVhMpDZ2FzV+dQUvSlI+nhIaJnjYpt1cPNrJo9Skx5..." }
```

5. The 5 Security Verification Gates

When a license key is submitted via the Web UI or validated by backend API endpoints (/api/license/validate, /api/license/activate), it must pass **5 consecutive security verification gates** in sequence:

Gate	Verification Check	Failure Reason / Security Rule
Gate 1	**Cryptographic Signature Check**	Verifies RSA / Ed25519 signature against vendor public key. Fails if key is tampered or forged.
Gate 2	**Hardware Fingerprint Match**	Computes current server hardware fingerprint & MAC addresses. Fails if license was generated for a different machine.
Gate 3	**Expiration Date Check**	Compares UTC current timestamp against Expiry timestamp. Fails if subscription period has ended.
Gate 4	**SHA-256 Checksum Integrity**	Re-calculates payload SHA-256 hash digest to confirm internal data structure claims match.
Gate 5	**Database & Session Enforcer**	Validates active license state in toolcrib.license_info database table on every authenticated API request.

6. End-to-End License Activation Workflow

Step 1: Machine Request File Generation (Client System)
• Execute: python License/client_license.py --request
• Output: Generates License/machine_request.txt containing MAC address and 64-character SHA-256 Fingerprint.

Step 2: License Issuance (Software Vendor)
• Vendor runs: python License/generate_license.py --request License/machine_request.txt --id TC-10001 --days 365
• Output: Generates RSA signed License/license_TC-10001.json.

Step 3: Web UI Validation & Activation (Admin User)
• Admin opens ToolCrib Web UI (http://localhost:3000).
• Pastes **entire JSON block** (including LicenseId, Fingerprint, Product, Expiry, Signature) into input field.
• Clicks ■ **Validate License** -> Calls POST /api/license/validate.
• Clicks ■ **Activate License** -> Calls POST /api/license/activate.

Step 4: Database Storage & State Update
• Deactivates previous licenses in toolcrib.license_info.
• Registers company details in toolcrib.companies.
• Writes new license record with active status, SHA-256 hash, and RSA signature.